

10-Triton-Inference-Server

1 Triton Inference Server

Document Version: 1.0

Generated: December 4, 2025

Standard: NVIDIA Triton Model Repository Format

Status: Industry Standard (NVIDIA)

1.1 Overview

Triton Inference Server (formerly TensorRT Inference Server) is NVIDIA’s production inference platform. It optimizes model serving across diverse hardware (GPUs, CPUs, edge devices).

1.1.1 Key Features

- **Multi-Framework:** TensorFlow, PyTorch, ONNX, TensorRT, Triton Backend
- **Multi-GPU:** Parallel execution across GPUs
- **Model Repository:** Standardized directory layout
- **Dynamic Batching:** Automatic batching for throughput
- **Ensemble Models:** Chained model pipelines
- **Metrics API:** Prometheus metrics
- **In-Process Backend:** C++ plugin system

1.1.2 Primary Use Cases

1. **High-Throughput Inference:** GPU-optimized serving
 2. **Multi-Model Management:** Serving many models simultaneously
 3. **Ensemble Pipelines:** Chained model inference
 4. **Real-Time Optimization:** Dynamic batching
 5. **Edge Deployment:** NVIDIA Jetson deployment
-

1.2 Authoritative References

- **Triton Docs:** <https://docs.nvidia.com/deeplearning/triton>
 - **GitHub Repository:** <https://github.com/triton-inference-server/server>
 - **Model Repository Format:** https://docs.nvidia.com/deeplearning/triton/user-guide/docs/user_guide/model_repository.html
 - **Backend Documentation:** https://docs.nvidia.com/deeplearning/triton/user-guide/docs/reference/backend_apis.html
-

1.3 Format Structure

1.3.1 Model Repository Layout

Real example:

```

model_repository/
├── bert_classifier/
│   ├── 1/                                # Model version
│   │   ├── model.pt                      # PyTorch model
│   │   └── tokenizer.json
│   ├── config.pbtxt                      # Model configuration
│   └── labels.txt
├── image_classifier/
│   ├── 1/
│   │   ├── model.onnx                    # ONNX model
│   │   ├── resnet50_labels.txt
│   └── config.pbtxt
├── preprocessing_ensemble/
│   ├── 1/
│   │   ├── preprocessing.pt
│   │   ├── config.pbtxt
│   │   └── ensemble_model.onnx          # Ensemble definition
└── ensemble_pipeline/
    ├── 1/
    └── config.pbtxt                      # Ensemble orchestration

```

1.3.2 Model Configuration (config.pbtxt)

Real example (BERT PyTorch model):

```

name: "bert_classifier"
platform: "pytorch_libtorch"
max_batch_size: 32

# Model input specification
input [
  {
    name: "input_ids"
    data_type: TYPE_INT64
    reshape { shape: [ -1, 128 ] }
    optional: false
  },
  {
    name: "attention_mask"
    data_type: TYPE_INT64
    reshape { shape: [ -1, 128 ] }
    optional: false
  },
  {
    name: "token_type_ids"
    data_type: TYPE_INT64
    reshape { shape: [ -1, 128 ] }
    optional: false
  }
]

# Model output specification
output [
  {
    name: "logits"
    data_type: TYPE_FP32
    reshape { shape: [ -1, 2 ] }
    label_filename: "labels.txt"
  }
]

# Dynamic batching
dynamic_batching {
  preferred_batch_size: [ 16, 32 ]
  max_queue_delay_microseconds: 100000
  preserve_ordering: false
}

```

```

}

# Optimization settings
optimization {
  graph { execution_accelerators {
    gpu_execution_accelerator {
      gpu_device: 0
    }
  } }
}

# Instance groups (GPU placement)
instance_group [
  {
    name: "bert_gpu_0"
    kind: KIND_GPU
    gpus: [ 0 ]
    count: 2
  },
  {
    name: "bert_gpu_1"
    kind: KIND_GPU
    gpus: [ 1 ]
    count: 2
  }
]

# Resource limits
default_model_filename: "model.pt"
version_policy {
  policy: "latest"
}

# Response cache (optional)
response_cache {
  enable: true
}

```

1.3.3 Model Configuration for TensorFlow

```

name: "tf_image_classifier"
platform: "tensorflow_savedmodel"
max_batch_size: 64

input [
  {
    name: "input_images"
    data_type: TYPE_UINT8
    dims: [ -1, 224, 224, 3 ]
    reshape { shape: [ -1, 224, 224, 3 ] }
  }
]

output [
  {
    name: "predictions"
    data_type: TYPE_FP32
    dims: [ -1, 1000 ]
    label_filename: "imagenet_labels.txt"
  }
]

dynamic_batching {
  preferred_batch_size: [ 32, 64 ]
  max_queue_delay_microseconds: 50000
}

instance_group [
  {
    kind: KIND_GPU
    count: 1
  }
]

```

1.3.4 Ensemble Model Configuration

```

name: "preprocessing_inference_postprocessing"
platform: "ensemble"

# Input specification (ensemble input)
input [
  {
    name: "image_b64"
    data_type: TYPE_STRING
  }
]

# Output specification (final output)
output [
  {
    name: "class_names"
    data_type: TYPE_STRING
  }
]

# Ensemble composition
ensemble_scheduling {
  step [
    {
      model_name: "image_preprocessor"
      model_version: -1
      input_map {
        key: "image_input"
        value: "image_b64"
      }
      output_map {
        key: "preprocessed_image"
        value: "preprocessed"
      }
    },
    {
      model_name: "image_classifier"
      model_version: -1
      input_map {
        key: "images"
        value: "preprocessed"
      }
      output_map {
        key: "logits"
        value: "classification_output"
      }
    },
    {
      model_name: "postprocessor"
      model_version: -1
      input_map {
        key: "logits_input"
        value: "classification_output"
      }
      output_map {
        key: "class_names"
        value: "class_names"
      }
    }
  ]
}

```

1.4 Procedural Use-Cases

1.4.1 Use-Case 1: PyTorch Model Deployment

Step 1: Create Model Directory:

```
mkdir -p model_repository/bert_classifier/1
```

Step 2: Save Model:

```
import torch
from transformers import AutoModel, AutoTokenizer

model = AutoModel.from_pretrained("bert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Convert to TorchScript
scripted_model = torch.jit.script(model)
scripted_model.save("model_repository/bert_classifier/1/model.pt")

# Save tokenizer
import json
with open("model_repository/bert_classifier/1/tokenizer.json", "w") as f:
    json.dump(tokenizer.get_vocab(), f)
```

Step 3: Create config.pbtxt:

```
cat > model_repository/bert_classifier/config.pbtxt << 'EOF'
name: "bert_classifier"
platform: "pytorch_libtorch"
max_batch_size: 32

input [
  {
    name: "input_ids"
    data_type: TYPE_INT64
    reshape { shape: [ -1, 128 ] }
  }
]

output [
  {
    name: "logits"
    data_type: TYPE_FP32
    reshape { shape: [ -1, 2 ] }
  }
]

dynamic_batching {
  preferred_batch_size: [ 16, 32 ]
  max_queue_delay_microseconds: 100000
}

instance_group [
  {
    kind: KIND_GPU
    count: 1
  }
]
EOF
```

Step 4: Launch Triton:

```
docker run --gpus all -p 8000:8000 -p 8001:8001 -p 8002:8002 \
-v $(pwd)/model_repository:/models \
nvr.io/nvidia/tritonserver:23.04-py3 \
tritonserver --model-repository=/models
```

1.4.2 Use-Case 2: Client Inference

Python Triton Client:

```

import tritonclient.http as httpclient
import numpy as np
from tritonclient.utils import np_to_triton_dtype

class TritonInferenceClient:
    def __init__(self, url: str = "localhost:8000"):
        self.client = httpclient.InferenceServerClient(url)

    def predict(
        self,
        model_name: str,
        inputs: dict,
        model_version: str = "1"
    ) -> dict:
        """Perform inference on Triton server"""

        # Prepare inputs
        triton_inputs = []
        for name, data in inputs.items():
            input_data = httpclient.InferInput(
                name,
                data.shape,
                np_to_triton_dtype(data.dtype)
            )
            input_data.set_data_from_numpy(data)
            triton_inputs.append(input_data)

        # Prepare outputs
        triton_outputs = [
            httpclient.InferRequestedOutput(
                "logits",
                binary_data=False
            )
        ]

        # Perform inference
        response = self.client.infer(
            model_name=model_name,
            inputs=triton_inputs,
            outputs=triton_outputs,
            model_version=model_version
        )

        # Extract outputs
        logits = response.as_numpy("logits")

        return {
            "logits": logits,
            "model_name": response.get_response()["model_name"],
            "model_version": response.get_response()["model_version"]
        }

# Usage
client = TritonInferenceClient()

# Prepare input
input_ids = np.array([[101, 2054, 2003, 2009, 102]], dtype=np.int64)

# Run inference
result = client.predict("bert_classifier", {"input_ids": input_ids})
print(f"Logits shape: {result['logits'].shape}")
print(f"Predictions: {result['logits']}")

```

gRPC Client (C++):


```

#include <grpc++/grpc++.h>
#include "grpc_service.pb.h"

class TritonGrpcClient {
public:
    TritonGrpcClient(const std::string& url) {
        channel_ = grpc::CreateChannel(
            url,
            grpc::InsecureChannelCredentials()
        );
        stub_ = inference::GRPCInferenceService::NewStub(channel_);
    }

    void Infer(const std::string& model_name) {
        inference::ModelInferRequest request;
        request.set_model_name(model_name);
        request.set_model_version("1");

        // Add input
        auto* input = request.add_inputs();
        input->set_name("input_ids");
        input->set_datatype("INT64");
        input->add_shape(1);
        input->add_shape(128);

        // Set input data
        std::vector<int64_t> data = {101, 2054, 2003, 2009, 102};
        input->set_raw_contents(
            std::string((char*)data.data(), data.size() * sizeof(int64_t))
        );

        // Execute
        inference::ModelInferResponse response;
        grpc::ClientContext context;

        grpc::Status status = stub_->ModelInfer(
            &context,
            request,
            &response
        );

        if (!status.ok()) {
            std::cerr << "RPC failed: " << status.error_message() << std::endl;
            return;
        }

        std::cout << "Model: " << response.model_name() << std::endl;
    }

private:
    std::shared_ptr<grpc::Channel> channel_;
    std::unique_ptr<inference::GRPCInferenceService::Stub> stub_;
};

```

1.4.3 Use-Case 3: Ensemble Pipeline

Create Preprocessing Model (Python):

```

import torch
import torch.nn as nn
from PIL import Image
import base64
from io import BytesIO
import numpy as np

class ImagePreprocessor(nn.Module):
    def forward(self, image_b64_tensor):
        # Decode, resize, normalize
        # This runs in TorchScript
        pass

# Save as TorchScript
preprocessor = ImagePreprocessor()
torch.jit.script(preprocessor).save(
    "model_repository/preprocessing/1/model.pt"
)

```

Ensemble Configuration:

```

name: "image_pipeline"
platform: "ensemble"

input [
  {
    name: "image_b64"
    data_type: TYPE_STRING
  }
]

output [
  {
    name: "class_name"
    data_type: TYPE_STRING
  }
]

ensemble_scheduling {
  step [
    {
      model_name: "image_preprocessor"
      input_map { key: "input" value: "image_b64" }
      output_map { key: "output" value: "preprocessed" }
    },
    {
      model_name: "resnet50_classifier"
      input_map { key: "images" value: "preprocessed" }
      output_map { key: "logits" value: "logits" }
    },
    {
      model_name: "label_converter"
      input_map { key: "logits" value: "logits" }
      output_map { key: "labels" value: "class_name" }
    }
  ]
}

```

1.5 Examples

1.5.1 Example 1: Docker Compose Setup

```

version: '3.8'

services:
  triton:
    image: nvcr.io/nvidia/tritonserver:23.04-py3
    ports:
      - "8000:8000" # HTTP
      - "8001:8001" # gRPC
      - "8002:8002" # Metrics
    volumes:
      - ./model_repository:/models
    environment:
      - NVIDIA_VISIBLE_DEVICES=all
      - TRITON_LOG_LEVEL=2
    command: tritonserver --model-repository=/models
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]

  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin

```

1.5.2 Example 2: Metrics Collection

Access Prometheus Metrics:

```
curl http://localhost:8002/metrics
```

Real Metrics Output:

```

# HELP nv_inference_request_total Total number of inference requests
# TYPE nv_inference_request_total counter
nv_inference_request_total{model="bert_classifier",version="1"} 1523.0

# HELP nv_inference_exec_time_ms Inference execution time in ms
# TYPE nv_inference_exec_time_ms histogram
nv_inference_exec_time_ms_bucket{le="50.0",model="bert_classifier"} 1200.0
nv_inference_exec_time_ms_bucket{le="100.0",model="bert_classifier"} 1450.0

# HELP nv_gpu_utilization GPU utilization percentage
# TYPE nv_gpu_utilization gauge
nv_gpu_utilization{gpu_id="0"} 87.5
nv_gpu_utilization{gpu_id="1"} 92.3

```

1.5.3 Example 3: Custom Backend

C++ Backend Plugin:

```
#include "triton/core/tritonserver.h"

TRITONSERVER_Error* CustomBackendInitialize(TRITONSERVER_Backend* backend) {
    // Initialize backend
    return nullptr;
}

TRITONSERVER_Error* CustomBackendModelInstanceExecute(
    TRITONSERVER_ModelInstance* instance,
    TRITONSERVER_InferenceRequest* request) {

    // Extract inputs
    // Run custom Logic
    // Set outputs

    return nullptr;
}
```

1.6 Performance Tuning

Parameter	Impact	Use Case
max_batch_size	Throughput vs latency	High-throughput scenarios
dynamic_batching	Automatic batching	Variable input rate
instance_group	GPU placement	Multi-GPU scaling
cache	Response caching	Repeated requests
rate_limiter	Request throttling	Resource protection

1.7 Tools & Ecosystem

Tool	Description	URL
Triton Server	Inference server	https://github.com/triton-inference-server
TensorRT	Model optimization	https://developer.nvidia.com/tensorrt
NVIDIA NGC	Model hub	https://catalog.ngc.nvidia.com
Triton Metrics	Prometheus integration	https://docs.nvidia.com/deeplearning/triton/user-guide
Model Analyzer	Performance profiling	https://github.com/triton-inference-server/model_analyzer

Navigation: [Back to Index](#) | [Previous: KServe](#) | [Next: Vector Embeddings Standards](#)